

MicroLA SIMD Optimization Strategy

SIMD Acceleration Tiers

«Rectangle»
Tier 3: RISC-V V Extensions

CONFIG_MICROLA_RISCV

Platforms:

- RISC-V processors with V extensions
- SiFive U74, Alibaba XuanTie C910

Performance:

- Vec2/3/4 operations: 1.3-2× faster
- Matrix 3×3/4×4: 2-3× faster
- Variable-length vectors (VL)
- Optimized for low-power cores

Implementation:

RISC-V intrinsics are used directly in vector.hpp, matrix.hpp, and quaternion.hpp (NOT in simd_helpers.hpp).

Intrinsics:

vsetvl_e32m1 (set vector length)
vle32_v_f32m1, vse32_v_f32m1 (load/store)
vfadd_vv_f32m1, vsub_vv_f32m1 (add/sub)
vfmul_vv_f32m1, vfmul_vf_f32m1 (multiply)
vfredsum_vs_f32m1_f32m1 (reduction)

«Rectangle»
Tier 1b: x86 AVX/AVX2

CONFIG_MICROLA_AVX

Platforms:

- x86_64 (Intel/AMD)
- AVX/AVX2 capable CPUs

Performance / Features:

- Wide 256-bit loads/stores (AVX)
- `\_mm\_i32gather\_ps` (AVX2) for indexed loads — used in 4-element gathers for inner kernels
- FMA via `vfmadd\*` / `\_mm\_fmadd\_ps`
- High throughput for batched vector/matrix ops

«Rectangle»
Tier 1: ARM NEON

CONFIG_MICROLA_NEON

Platforms:

- ARM Cortex-A (Raspberry Pi, phones)
- Apple Silicon (M1/M2/M3)
- ARMv64 (aarch64)

Performance:

- Vec3 operations: 1.4× faster
- Quaternion multiply: 3.5× faster
- Matrix 3×3/4×4: 3-4× faster
- FMA instructions enabled
- Fast reciprocal/rsqrt

Intrinsics:

vld1q_f32, vaddq_f32,
vsubq_f32, vmulq_f32,
vmaq_f32 (FMA),
vrecpeq_f32, vrsqrteq_f32

«Rectangle»
Tier 2: CMSIS-DSP

CONFIG_MICROLA_CMSIS

Platforms:

- ARM Cortex-M4F/M7/M33/M85
- Microcontrollers (STM32, NXP, etc.)

DSP Functions:

Vector ops:
arm_add_f32, arm_sub_f32
arm_scale_f32, arm_negate_f32
arm_dot_prod_f32, arm_sqrt_f32

Matrix ops:
arm_mat_add_f32, arm_mat_sub_f32
arm_mat_scale_f32, arm_mat_mult_f32
arm_mat_trans_f32

Quaternion ops:
arm_sqrt_f32, arm_scale_f32
arm_negate_f32 (conjugate)

Performance:

- Vec operations: 1.3-1.8× faster
- Matrix 3×3/4×4: 2-3× faster
- Quaternion normalize: 1.8× faster
- Power-efficient DSP instructions

Benefits:

- Vendor-optimized for ARM Cortex-M
- DSP instruction utilization
- Hardware MAC (Multiply-Accumulate)
- Lower power consumption
- Float/Fixed-point support

«Rectangle»
Tier 4: Generic C++

Portable Scalar

Platforms:

- All C++17 compilers
- Fallback when no SIMD enabled

Implementation:

- std::fill_n, std::copy_n for bulk ops
- Plain scalar loops
- Compiler auto-vectorization friendly
- constexpr compatible
- Branch prediction hints

Always Available

Default for all platforms

Vec<float, N> Optimization

N=4: NEON float32x4_t / AVX __m128

N=2: NEON float32x2_t / AVX __m128

N=3: NEON float32x4_t (pad) / AVX __m128

Optimized Layout:

Eliminated temporary arrays
Direct SIMD register operations
AVX via simd_helpers.hpp
NEON via direct intrinsics
1.4× faster than scalar

Quaternion<float> Optimization

Full NEON multiply

Vec3 integration

SLERP optimized

NEON Quaternion Product:

3.5× faster than scalar
AVX via simd_helpers.hpp
Uses vfmag_f32 (FMA)
Minimal register pressure

Matrix Operations

Block operations

4×4 multiply

3×3 multiply

Matrix SIMD:

3-4× speedup for 3×3, 4×4
Row-major optimized
Cache-friendly access

